

Министерство науки и высшего образования Российской Федерации
Муромский институт (филиал)
федерального государственного бюджетного образовательного учреждения высшего образования
**«Владимирский государственный университет
имени Александра Григорьевича и Николая Григорьевича Столетовых»**
(МИ ВлГУ)

Факультет информационных технологий и радиоэлектроники
Кафедра информационных систем

КУРСОВАЯ РАБОТА

по курсу Прикладная разработка на Java
на тему: Разработка программы «морской бой»

Руководитель

К. Т. Н.

(уч. степень, звание)

Метелкин А. С.

(фамилия, инициалы)

(подпись)

(дата)

Члены комиссии

Студент ИС - 122
(группа)

Жолнин А.А.

(фамилия, инициалы)

(подпись)

(дата)

(подпись)

(Ф.И.О.)

(подпись)

(Ф.И.О.)

Муром 2025

В курсовой работе было разработано игровое приложение Морской бой. Приложение позволяет игроку выбрать из 2-х режимов игры: PvE, PvP. Пользователь может начать полноценную игру против компьютера, или же сыграть против другого пользователя, а после просматривать статистику игр, общее количество побед, поражений и исходы других игр. В качестве среды разработки был выбран IntelliJ IDEA, язык программирования Java, графический интерфейс реализован на JavaFX.

СОДЕРЖАНИЕ

Введение.....	4
1 Анализ технического задания.....	5
1.2 Описание функционала.....	5
1.3 Выбор технологии разработки приложения.....	6
2 Проектирование программы.....	9
2.1 Архитектура игрового приложения.....	9
2.2 Описание модулей системы	9
2.2.1 Модуль кораблей.....	9
2.2.2 Модуль игровой логики.....	10
2.2.3 Модуль базы данных.....	11
2.2.4 Модуль пользовательского интерфейса.....	11
2.3 Взаимодействие модулей.....	13
3 Разработка программы.....	15
3.1 Реализация структуры координат.....	15
3.2 Структура игроков.....	15
3.3 Реализация игровых объектов.....	15
3.4 Разработка главного меню.....	19
3.5 Реализация хранения и отображения статистика.....	20
3.6 Взаимодействие всех классов.....	20
4 Тестирование.....	23
4.1 Тестирование игрового процесса.....	23
4.2 Тестирование сохранения и отображения статистики.....	26
4.3 Итоги тестирования.....	29
Заключение.....	30
Список литературы.....	31

					МИВУ.09.03.02-00.000					ПЗ				
Изм	Лист	№ докум.	Подп.	Дата	Разработка программы «морской бой»					Лит.	Лист	Листов		
Студент	Жолнин А.А.									у		3	31	
Руков.	Метелкин А.С.									МИ ВлГУ ИС-122				
Конс.														
Н.контр.														
Зав.каф.														

Введение

Игра "Морской бой" является классической стратегической игрой, которая развивает логическое мышление и тактические навыки. Целью данной курсовой работы является разработка программного обеспечения для игры "Морской бой" с поддержкой режимов "Игрок против компьютера" (PvE) и "Игрок против игрока" (PvP). Приложение реализовано на языке Java с использованием графической библиотеки JavaFX для создания интуитивно понятного интерфейса. В качестве базы данных выбрана SQLite для хранения истории игр и статистики игроков.

Тема данной курсовой работы – разработка игрового приложения морской бой.

Целью курсовой работы является закрепление знаний, связанных с разработкой на языке java, используемых базовых паттернов программирования, интеграции приложения и базы данных, а также получение новых знаний и навыков при использовании графической библиотеки JavaFX.

Проанализировав цель работы, можно выделить несколько задач:

1. Анализ технического задания
2. Проектирование программы
3. Разработка программы
4. Тестирование
5. Отладка и тестирование приложения

В результате выполнения всех поставленных задач будет разработано готовое приложение, реализующее первоначальную задумку.

1 Анализ технического задания

По заданию необходимо создать программу, реализующую принципы работы игры морской бой. В качестве языка программирования был выбран Java, а среда разработки – IntelliJ IDEA. Использование графической библиотеки JavaFX позволит создать интерфейс игры.

1.1 Описание функционала

Сначала разберемся, что такое морской бой. Морской бой – это стратегическая игра с неполной информацией, в которой два игрока по очереди атакуют клетки на поле 10×10 , стремясь потопить флот противника. Флот состоит из кораблей разного размера: однопалубные (1 клетка), двухпалубные (2 клетки), трехпалубные (3 клетки) и четырехпалубный (4 клетки). За ход игрок выбирает одну клетку для выстрела, обозначая результат: «попал» (если задел корабль), «мимо» (если клетка пуста) или «потопил» (если корабль полностью уничтожен). Корабли размещаются горизонтально или вертикально, не соприкасаясь друг с другом, даже углами. Цель – первым потопить все корабли противника.

Правила взаимодействия следующие: игроки не видят расположение кораблей соперника, пока не произведут выстрел. Попадание фиксируется, если выстрел задевает корабль, а если корабль полностью поражен, он отмечается как потопленный в предварительном просмотре. Если выстрел не попадает в корабль, клетка помечается как пустая. Игра продолжается по очереди, пока один из игроков не потопит весь флот противника.

Дополнительные аспекты игры:

1. Размещение флота. Перед началом игры каждый игрок располагает свои корабли на поле, следуя правилу, что они не должны соприкасаться.

2. Предварительный просмотр. После потопления корабля игрок может пометить его на своей карте, чтобы лучше анализировать оставшиеся цели.

3. Конец игры. Игра завершается, когда один из игроков теряет все корабли. Ничья возможна, если оба игрока одновременно топят последние корабли друг друга.

Морской бой требует тактического мышления и умения предугадывать расположение кораблей противника на основе результатов выстрелов.

Исходя из вышеперечисленного, можно подвести итоги того, что должна выполнять программа и какие к ней можно привести требования:

- Расстановка кораблей вручную.
- Игра против компьютера с интеллектуальным алгоритмом атаки.
- Локальная игра для двух игроков.
- Сохранение истории боев и статистики побед/поражений .
- Просмотр сыгранных партий.
- Просмотр статистики игрока

1.2 Выбор и сравнения технологий разработки приложения

Вообще изначально приходилось выбирать между библиотекой Swing и JavaFX одной из причин стало удобство работы с JavaFX, но так как удобство понятие субъективное то основными причинами выбора JavaFX стало это, то что при работе с БД JavaFX имеет несколько преимуществ, которые делают его предпочтительным в современных приложениях, особенно для интерфейсов, взаимодействующих с БД.

JavaFX поддерживает встроенные механизмы для асинхронной работы, такие как Task и Service, что критически важно при взаимодействии с БД.

Запросы к БД (особенно длительные) могут блокировать UI, и JavaFX позволяет легко выполнять их в фоновом потоке, обновляя интерфейс через Platform.runLater или свойства привязки (bindings). А Swing использует более старые компоненты, такие как JTable, которые требуют ручного управления моделью данных (например, через TableModel). Это менее удобно для динамического обновления данных из БД. Это стало ключевой причиной выбора JavaFX

Так же наличие Аппаратного ускорение в JavaFX использующая GPU для рендеринга, что делает его более производительным для современных приложений с графикой, анимациями и мультимедиа в отличии от Swing.

И причина по которой я считаю, что JavaFX удобнее Swing это потому что JavaFX имеет инструменты вроде Scene Builder, которые упрощают проектирование интерфейсов, в отличие от Swing, где таких инструментов меньше, а так же наличие поддержки FXML в JavaFX, которая позволяет использовать FXML для декларативного описания интерфейса, что разделяет логику и дизайн, упрощая разработку и поддержку кода.

Так JavaFX лучше AWT, потому что предлагает современный и богатый интерфейс с поддержкой CSS, FXML, анимаций, 3D-графики и визуальных эффектов, тогда как AWT использует устаревшие системно-зависимые компоненты с ограниченной кастомизацией. JavaFX обеспечивает более гибкую и кросс-платформенную разработку с аппаратным ускорением, в отличие от AWT, который зависит от нативных элементов ОС и выглядит по-разному на Windows, Linux и macOS. Кроме того, JavaFX поддерживает привязку данных, Observable коллекции и реактивное программирование, что упрощает разработку сложных интерфейсов, в то время как AWT работает в однопоточном режиме (Event Dispatch Thread) и не предоставляет таких возможностей. В целом, JavaFX — это более мощный, современный и удобный фреймворк для создания графических приложений на Java.

В качестве способа хранения данных была выбрана база данных SQLite. Так как SQLite часто оказывается удобнее Firebird и PostgreSQL в случаях, когда нужна легкая и самодостаточная база данных без лишних сложностей. Главное преимущество SQLite — отсутствие необходимости в серверной части. В отличие от Firebird и PostgreSQL, которые требуют установки, настройки и постоянного обслуживания сервера, SQLite работает прямо с файлом на диске. Это делает его идеальным для встраиваемых систем, мобильных приложений и небольших программ, где важны простота развертывания и минимальные требования к ресурсам.

Еще одно ключевое преимущество SQLite — высокая скорость работы с локальными данными. Поскольку здесь нет сетевых задержек и накладных расходов на клиент-серверное взаимодействие, операции чтения и записи выполняются быстрее в однопользовательских сценариях. Firebird и PostgreSQL, конечно, мощнее в плане многопоточности и сложных запросов, но для простых задач их использование может быть избыточным. SQLite потребляет меньше памяти и процессорного времени, что делает его отличным выбором для устройств с ограниченными ресурсами.

Наконец, SQLite проще в переносе и интеграции. Вся база данных хранится в одном файле, который можно свободно копировать и перемещать без дополнительных инструментов. В то время как Firebird и PostgreSQL требуют дампов или специальных утилит для миграции данных.

2 Проектирование программы

2.1 Архитектура игрового приложения

На этапе проектирования были определены базовые модули для грамотного функционирования системы. В основу проектирования лег принцип разделения ответственности, каждый модуль выполняет строго отведенную ему функцию.

Система реализует трехуровневую архитектуру

1. Уровень представления – отвечает за взаимодействие пользователя с графическим интерфейсом.
2. Уровень логики – отвечает за реализацию основных правил игры в шахматы.
3. Уровень данных – отвечает за сохранение и загрузку игровых данных.

2.2 Описание модулей системы

2.2.1 Модуль кораблей

На начальном этапе проектирования системы морского боя было решено начать с кораблей, так как они составляют основу игровой логики и определяют большинство событий в игре.

Модуль кораблей реализован через класс Ship, который лежит в основе игровой механики. Для упрощения разработки был создан единый класс, представляющий корабли разных размеров (от 1 до 4 клеток). Этот класс определяет базовые свойства и поведение кораблей, а конкретные

характеристики (размер, количество попаданий) задаются при создании экземпляра.

Модуль кораблей включает:

- Класс Ship: Определяет корабль с атрибутами size (размер корабля, от 1 до 4 клеток) и hits (количество попаданий). Методы включают:
- getSize(): Возвращает размер корабля.
- hit(): Увеличивает счетчик попаданий.
- isSunk(): Проверяет, потоплен ли корабль (если количество попаданий равно размеру).

Класс Ship используется в связке с классом Board, где хранится информация о расположении кораблей (координаты и ориентация), и классом GameController, который управляет их размещением и обработкой атак. Этот модуль критически важен, так как он отвечает за логику состояния кораблей (целый, поврежденный, потопленный), что является основой игрового процесса морского боя. На этапе разработки будут дополнительно рассмотрены методы и их взаимодействие с другими модулями, такими как отображение потопленных кораблей с предварительным просмотром в интерфейсе.

2.2.2 Модуль игровой логики

Второй модуль системы – логика морского боя, отвечающая за реализацию основных правил и механики игры, что делает её ядром программы.

Модуль охватывает следующие аспекты:

- Создание и управление игровым полем: Формируется поле 10×10 клеток, где отслеживаются состояния клеток (пустая, корабль,

попадание, промах, потопленный корабль). Поддерживается размещение флота, состоящего из 4 однопалубных, 3 двухпалубных, 2 трехпалубных и 1 четырехпалубного корабля, с проверкой корректности их расположения (без соприкосновений и выхода за границы поля).

- Логика размещения кораблей: Реализовано как ручное размещение, так и автоматическое для компьютерного игрока. Учитывается ориентация кораблей (вертикальная или горизонтальная) и запрет на их пересечение или касание.
- Логика выстрелов: Обрабатываются атаки с фиксацией результатов: промах (синий цвет), попадание (красный цвет) или потопление корабля (бордовый цвет). Проверяется, не была ли клетка атакована ранее, и обновляется состояние корабля после попадания.
- Смена хода: В режиме PvP ход переходит к другому игроку после промаха, а в PvE – между игроком и компьютером. Компьютер применяет два подхода: случайные выстрелы или "охота" (атака соседних клеток после попадания).
- Проверка окончания игры: Определяется, потоплены ли все корабли одного из игроков, что завершает игру.
- Валидация действий: Проверяется допустимость выстрелов (в пределах поля, без повторных атак) и размещения кораблей (отсутствие пересечений, соблюдение границ). При некорректных действиях выдаются сообщения об ошибках.
- Поддержка интерфейса: Модуль обеспечивает отображение состояния поля, включая предварительный просмотр размещения кораблей (светло-зеленая подсветка для допустимых позиций, розовая для недопустимых) и визуализацию потопленных кораблей.

Этот модуль критически важен, так как гарантирует соблюдение правил морского боя и управляет игровым процессом. При проектировании ему уделялось особое внимание для обеспечения точности логики и взаимодействия с другими компонентами системы. В перспективе возможно расширение функционала, например, для записи и анализа истории боев, что позволит просматривать завершённые партии.

2.2.3 Модуль базы данных

Третий модуль отвечает за создание и работоспособность базы данных. При проектировании было решено использовать SQLite, и сохранять базу данных локально. Это решение позволит локально хранить базу данных без зависимости от сети и позволит просто разворачивать эту программу. Очевидно создание двух таблиц. Одна из них будет определять саму партию, в том числе id партии, имя игрока, режим(PvE, PvP), результат или победителя партии, дата сыгранной игры. Другая таблица будет определять статистику игрока, это обязательное условие для дальнейшей реализации просмотра статистики.

2.2.4 Модуль пользовательского интерфейса

Четвертый модуль системы отвечает за графическую реализацию игровой логики морского боя. Для создания интерфейса использована библиотека JavaFX, которая обеспечивает отображение сцен, кнопок и визуальных элементов. Модуль не монолитный и включает несколько компонентов, таких как игровое поле, главное меню и панель истории боев.

Игровое поле отображает два поля размером 10×10 клеток для игрока и противника. Клетки визуализируются с цветовой индикацией: белый цвет для пустых клеток, черный для кораблей, красный для попаданий, синий для промахов и бордовый для потопленных кораблей. Поддерживается предварительный просмотр размещения кораблей с подсветкой светло-зеленого цвета для допустимых позиций и розового для недопустимых, а также

возможность смены ориентации кораблей с помощью клавиши Z. Главное меню содержит стилизованные кнопки для выбора режима игры (PvP или PvE), просмотра истории боев и выхода из игры, оформленные на фоне изображения для удобной навигации. Панель истории боев позволяет просматривать записи сыгранных партий, включая имена игроков, победителя, режим игры и дату, представленные в виде таблицы, доступной из главного меню.

Интерактивные элементы обеспечивают удобство игры. При размещении кораблей предварительный просмотр помогает избежать ошибок, показывая допустимые позиции. Во время игры атаки по клеткам противника сопровождаются немедленным обновлением поля с соответствующей цветовой индикацией. После завершения партии, будь то победа или поражение, появляется диалоговое окно с опциями начать новую игру, вернувшись в главное меню, или выйти. В PvE-режиме компьютер автоматически размещает корабли, а игрок может атаковать поле противника, получая визуальную обратную связь о результатах выстрелов. В PvP-режиме поля переключаются между игроками, скрывая корабли противника до конца игры, когда все корабли раскрываются.

Для упрощения взаимодействия интерфейс откликается на действия игрока, блокируя повторные атаки по одной клетке и выводя сообщения об ошибках, например, о выстреле за пределы поля. Модуль пользовательского интерфейса делает процесс размещения кораблей, атак и анализа партий интуитивным и наглядным, уделяя особое внимание визуальной обратной связи, чтобы игроки могли легко ориентироваться в игровом процессе и принимать тактические решения.

2.3 Взаимодействие модулей

В данном разделе описывается взаимодействие модулей системы морского боя, представленное в виде последовательных этапов.

Сначала пользователь через пользовательский интерфейс выбирает режим игры (PvP, PvE или просмотр истории боев), что инициирует запуск соответствующего игрового процесса. Далее создается игровое поле размером 10×10 клеток с размещением кораблей (4 однопалубных, 3 двухпалубных, 2 трехпалубных, 1 четырехпалубный) с использованием модуля игровой логики, который взаимодействует с модулем кораблей для задания их свойств, таких как размер и ориентация. Пользователь выполняет действия, такие как размещение кораблей или атака, через модуль пользовательского интерфейса, который отображает поля, визуализирует состояния клеток (пустая, корабль, попадание, промах, потопленный) и обеспечивает предварительный просмотр размещения с цветовой подсветкой (светло-зеленая для допустимых позиций, розовая для недопустимых).

Модуль игровой логики обрабатывает действия, проверяя корректность размещения кораблей (отсутствие соприкосновений, соблюдение границ поля) и атак (попадание, промах, потопление), а также отслеживает ключевые события, такие как потопление корабля или завершение игры. После хода игрока в PvE-режиме управление передается модулю компьютерного противника, который выполняет атаку, используя случайные выстрелы или режим "охоты" при попадании. В PvP-режиме ход переходит к другому игроку. По окончании партии результат (победа или поражение) передается в модуль базы данных, который сохраняет информацию о бое, включая имена игроков, победителя, режим игры и дату, для последующего просмотра в истории боев.

В итоге спроектированная система включает самодостаточные модули, каждый из которых выполняет свои функции: пользовательский интерфейс обеспечивает взаимодействие, игровой движок реализует правила, модуль кораблей задает их поведение, компьютерный движок управляет действиями ИИ, а база данных сохраняет результаты. Разработанная программа полностью соответствует требованиям технического задания, обеспечивая логичное и эффективное взаимодействие компонентов

3 Разработка программы

3.1 Реализация структуры координат

Разработка программы началась с реализации структуры для представления координат на игровом поле. Эта структура, реализованная в виде класса, определяет позицию клетки на поле 10×10 и хранит две координаты: строку и столбец. Она включает методы для получения значений строки и столбца, а также метод сравнения координат двух позиций для проверки их совпадения. Дополнительно реализован метод вычисления хэш-кода на основе координат, обеспечивающий корректную работу в коллекциях. Для удобства записи игровых действий создан метод преобразования координат в строковый формат (например, «А-1»), используемый для сохранения ходов и взаимодействия с базой данных. Также предусмотрен обратный метод, который преобразует строковый формат обратно в объект координат, что упрощает загрузку данных из истории боев. Строковое представление координат возвращается в формате «буква-цифра» для удобного отображения.

3.2 Структура игроков

Для обозначения игроков использована простая структура с двумя значениями, представляющими стороны игры: игрок 1 и игрок 2 (или компьютер в PvE-режиме). Это обеспечивает четкое разделение участников без необходимости сложных перечислений.

3.3 Реализация игровых объектов

Так как игра включает корабли и их взаимодействие с полем, был создан интерфейс, определяющий базовое поведение игровых объектов. Интерфейс содержит методы для получения текущей позиции корабля на поле, установки новой позиции при размещении, проверки допустимости размещения (учитываются границы поля, ориентация и отсутствие соприкосновений с

другими кораблями) и получения визуального представления для отображения в интерфейсе. Также предусмотрен метод для получения состояния корабля (целый, поврежденный, потопленный), используемый при обработке атак. Дополнительные методы включают установку и проверку статуса размещения корабля, что важно для валидации игровых действий, таких как запрет на пересечение кораблей или их размещение вне поля. Эти методы обеспечивают гибкость и унифицированность при реализации логики кораблей, позволяя корректно обрабатывать их поведение в игре.

Разработка игровых объектов началась с создания класса, представляющего корабль, который является основным элементом игры морского боя. Этот класс, названный Ship, определяет поведение кораблей и их взаимодействие с игровой логикой. Он включает базовые свойства и методы, необходимые для управления состоянием кораблей.

Класс Ship содержит поле, определяющее размер корабля (от 1 до 4 клеток), и счетчик попаданий, отслеживающий количество атак по кораблю. Основной метод проверки состояния корабля определяет, допустимо ли его размещение на поле: корабль не должен пересекаться с другими кораблями, касаться их (включая соседние клетки) и выходить за границы поля 10×10. Также метод учитывает ориентацию (вертикальная или горизонтальная). Дополнительные методы включают увеличение счетчика попаданий при атаке и проверку, потоплен ли корабль (если количество попаданий равно размеру). Для интеграции с базой данных и интерфейсом предусмотрен метод, возвращающий строковое представление корабля, например, его размер или статус, используемый при сохранении истории боев. Метод проверки состояния размещения подтверждает, был ли корабль корректно установлен на поле, что важно для валидации игровых действий, таких как запрет на размещение кораблей вплотную друг к другу.

Этот класс обеспечивает унифицированную логику для кораблей всех размеров, позволяя эффективно управлять их размещением, атаками и состоянием в процессе игры.

Класс Board представляет игровое поле морского боя размером 10×10 клеток и управляет состоянием клеток и кораблей. Он содержит двумерный массив для хранения состояний клеток (0 — пустая, 1 — корабль, 2 — попадание, 3 — промах, 4 — потопленный корабль) и список для отслеживания информации о кораблях (их координаты, ориентация и состояние). Основной метод размещения кораблей проверяет возможность установки корабля заданного размера и ориентации, учитывая отсутствие пересечений, касаний с другими кораблями и выход за границы поля. Метод обработки атаки (receiveAttack) определяет результат выстрела (попадание, промах или потопление), обновляя состояние клетки и проверяя, потоплен ли корабль. Дополнительный метод автоматического размещения кораблей генерирует случайное расположение флота (4 однопалубных, 3 двухпалубных, 2 трехпалубных, 1 четырехпалубный). Метод проверки окончания игры возвращает, потоплены ли все корабли на поле. Также реализован метод доступа к текущему состоянию поля для отображения в интерфейсе. Этот класс критически важен для управления игровой логикой и обеспечения корректного взаимодействия кораблей и атак.

Класс Player представляет игрока в игре, управляющего своим полем и атаками по полю противника. Он содержит ссылки на собственное игровое поле и поле противника. Основной метод выполнения хода позволяет игроку атаковать указанную клетку на поле противника, возвращая результат (попадание или промах). Метод доступа к собственному полю предоставляет информацию о состоянии кораблей игрока для обработки атак со стороны противника или компьютера. Класс не хранит сложных состояний, а служит для упрощения взаимодействия между игроком и игровой логикой, передавая

действия в модуль доски. Этот класс обеспечивает базовую функциональность для человеческого игрока в PvP или PvE режимах.

Класс Computer реализует поведение компьютерного противника в PvE-режиме. Он содержит ссылки на собственное поле и поле игрока, а также использует очередь для хранения координат потенциальных целей в режиме "охоты". Основной метод выполнения хода (makeMoveUntilMiss) реализует два режима атаки: случайный выбор клетки (если нет активных целей) и последовательная атака соседних клеток после попадания (режим "охоты"). Метод добавления соседних клеток в очередь проверяет их валидность (в пределах поля и не атакованы ранее). Дополнительный метод проверки координат подтверждает их корректность. Класс также предоставляет доступ к своему полю для обработки атак игрока. Этот класс обеспечивает интеллектуальное поведение компьютера, делая игру в PvE-режиме динамичной и тактически интересной.

Класс GameController управляет общим игровым процессом, координируя взаимодействие между игроками, компьютером и интерфейсом. Он содержит ссылки на поля игроков, объекты игроков и компьютера, а также флаги для отслеживания текущего хода и этапа игры (размещение кораблей или атака). Основной метод обработки хода игрока (handlePlayerMove) проверяет валидность атаки, обновляет состояние поля и определяет, продолжить ли ход или передать его другому игроку (или компьютеру). Метод размещения кораблей (placePlayerShip) валидирует и фиксирует расположение кораблей, переходя к следующему кораблю или этапу игры. Метод сохранения результата игры формирует запись боя и передает её в базу данных. Дополнительные методы возвращают информацию о текущем размере корабля для размещения, режиме игры (PvP или PvE) и состоянии хода. Этот класс является центральным узлом, обеспечивающим синхронизацию всех игровых компонентов.

Класс `GameView` отвечает за графическое отображение игры с использованием `JavaFX`. Он содержит ссылки на объекты игроков, компьютера и контроллера, а также элементы интерфейса: два поля 10×10 (в виде сеток), метки игроков и индикатор ориентации кораблей. Основным методом инициализации (`initialize`) создает графическое поле с координатами (A–J, 1–10) и задает начальное состояние клеток. Метод обновления сетки (`updateGrid`) синхронизирует отображение клеток с их состоянием (пустая, корабль, попадание, промах, потопленный), используя цвета (белый, черный, красный, синий, бордовый). Метод предварительного просмотра (`showShipPreview`) подсвечивает допустимые (светло-зеленые) и недопустимые (розовые) позиции при размещении кораблей. Методы отображения сообщений и ошибок показывают диалоговые окна с результатами игры или предупреждениями. Дополнительные методы управляют кликабельностью полей, скрывают или раскрывают корабли в конце игры. Этот класс обеспечивает интуитивный и наглядный интерфейс для взаимодействия игрока с игрой.

3.4 Разработка главного меню

Класс `MainMenu` реализует главное меню игры, предоставляя пользователю доступ к основным функциям. Он содержит элементы интерфейса, такие как фоновое изображение и кнопки для выбора режима игры (PvP, PvE), просмотра истории боев и выхода. Основным методом отображения (`show`) создает сцену с центрированными кнопками, стилизованными в едином дизайне. Метод просмотра истории боев открывает таблицу с данными о прошлых партиях (имена игроков, победитель, режим, дата). Дополнительные методы запускают игру в выбранном режиме, передавая управление контроллеру игры. Этот класс служит точкой входа в игру, обеспечивая удобную навигацию и доступ к функционалу.

3.5 Реализация хранения и отображения статистики

Класс `BattleRecord` представляет запись одной игровой партии для сохранения в базе данных. Он содержит поля для хранения имен игроков, победителя, режима игры (PvP или PvE) и даты боя. Основной метод сохранения (`save`) записывает данные в базу данных SQLite, используя подготовленный SQL-запрос. Методы доступа возвращают значения полей для отображения в истории боев. Метод строкового представления (`toString`) формирует читаемый формат записи, например, «Игрок1 vs Игрок2 | Победитель: Игрок1 | Режим: PvP | Дата: ...». Этот класс обеспечивает структурированное хранение результатов партий, упрощая их последующий просмотр.

Класс `DatabaseManager` управляет взаимодействием с базой данных SQLite для хранения и получения игровых данных. Он содержит методы для инициализации базы (создание директории и таблиц `battle_history` и `player_stats`) и подключения к ней. Основной метод сохранения результата игры (`saveGameResult`) записывает данные о бое в таблицу `battle_history`. Метод получения истории боев (`getBattleHistory`) возвращает список всех записей для отображения в интерфейсе. Метод обновления статистики игроков (`updatePlayerStats`) увеличивает счетчик побед или поражений для указанных игроков. Дополнительный метод закрытия соединения (`closeConnection`) обеспечивает корректное завершение работы с базой. Этот класс гарантирует надежное хранение и доступ к данным, поддерживая функционал истории и статистики.

3.6 Взаимодействие всех классов

Взаимодействие классов в игре "Морской бой" представляет собой слаженную систему, где каждый компонент выполняет строго определённую роль, передавая данные и управление другим модулям. Начинается всё с класса `Coordinate`, который служит фундаментом для работы с координатами на поле.

Этот класс активно используется практически всеми остальными компонентами: Board проверяет границы поля, Ship определяет позиции кораблей, GameView преобразует координаты в понятный игроку формат (например, "B-5"), а DatabaseManager сохраняет их в виде строк для истории игр.

Когда игрок запускает игру через MainMenu, система создаёт экземпляры Player и, в случае PvE-режима, Computer. Оба этих класса работают с собственными экземплярами Board, где хранится состояние их полей. На этапе расстановки кораблей Player через интерфейс GameView выбирает позиции, а GameController проверяет их корректность, обращаясь к методам Board. Тот, в свою очередь, использует логику Ship, чтобы убедиться, что корабли не пересекаются, не соприкасаются и помещаются в границы поля. Computer в это же время автоматически размещает свои корабли, вызывая Board.autoPlaceShips(), который случайным образом распределяет корабли, соблюдая те же правила.

Как только корабли расставлены, управление переходит к GameController, который координирует ходы. Когда игрок совершает атаку, GameView передаёт координаты выстрела в GameController, а тот делегирует обработку Board противника (в PvE — полю Computer). Метод Board.receiveAttack() определяет, был ли это промах, попадание или потопление корабля, обновляет состояние клеток и, если корабль уничтожен, оповещает GameController. В PvE-режиме после хода игрока GameController активирует Computer.makeMoveUntilMiss(), где ИИ анализирует поле игрока, выбирая цели: сначала случайно, а после попадания — атакуя соседние клетки.

GameView постоянно синхронизируется с Board, чтобы отображать актуальное состояние поля: пустые клетки, корабли, попадания (красные клетки), промахи (синие) и потопленные суда (бордовые). Если игра завершается, GameController формирует запись (BattleRecord) и сохраняет её

через DatabaseManager, который обновляет базу данных SQLite. MainMenu может затем запросить историю боёв у DatabaseManager и отобразить её игроку.

Весь процесс построен так, что GameController выступает центральным "дирижёром": он получает ввод от GameView, передаёт данные в Board и Player/Computer, а затем возвращает результаты обратно в интерфейс. Классы Ship и Coordinate играют вспомогательную роль, обеспечивая валидацию и базовую логику, но их работа невидима для игрока — всё взаимодействие происходит через интуитивный интерфейс GameView и фоновые процессы GameController. Это создаёт чёткое разделение ответственности: например, Board не знает о существовании GameView, а Computer не взаимодействует напрямую с DatabaseManager, что делает систему гибкой и удобной для расширения.

4 Тестирование

Тестирование программного обеспечения (ПО) является неотъемлемой частью процесса разработки и обеспечивает высокое качество конечного продукта. Тестирование выполняется для проверки соответствия ПО требованиям, выявления ошибок, дефектов и недоработок, а также для проверки функциональности, производительности и безопасности.

4.1 Тестирование игрового процесса

В первую очередь было уделено внимание проверки инициализации поля (рисунок 1)

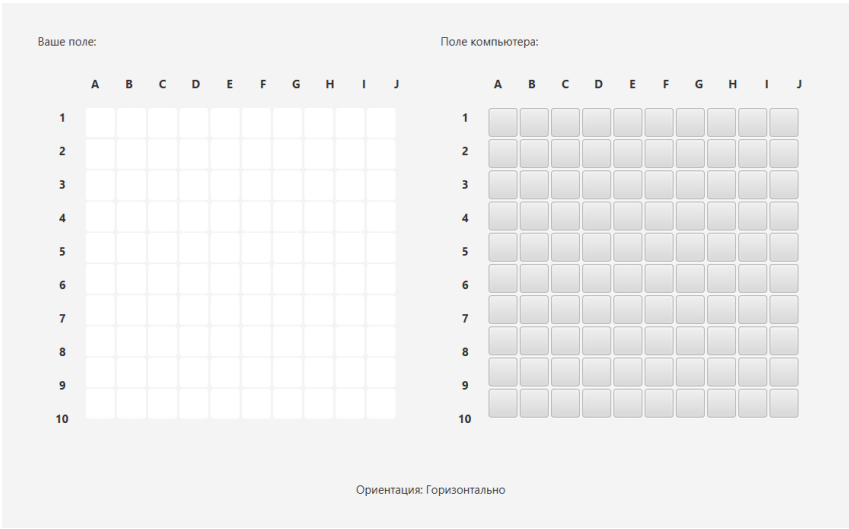


Рисунок 1 – Проверка начала игры

Также неотъемлемой частью сценария игры является размещения кораблей с их предварительным просмотром (рисунок 2), а так же скрывтие кораблей в PvP режиме (рисунок 3, 4)

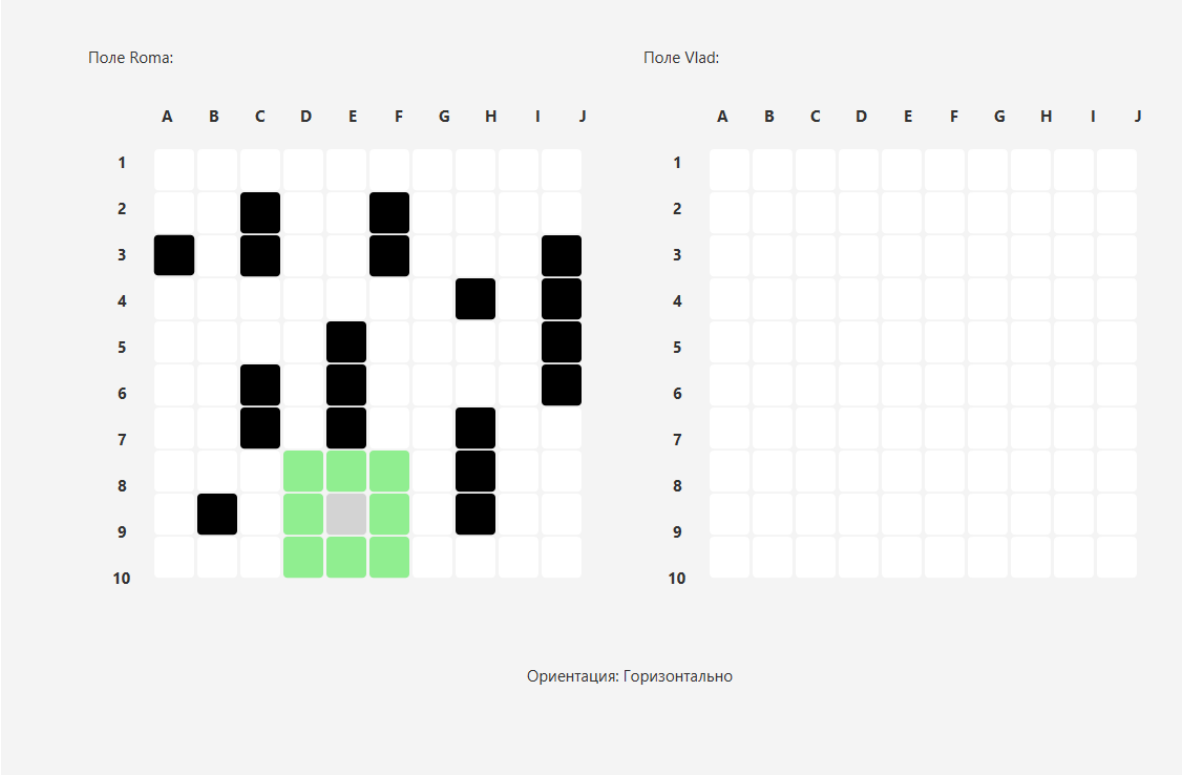


Рисунок 2 – Проверка размещения кораблей

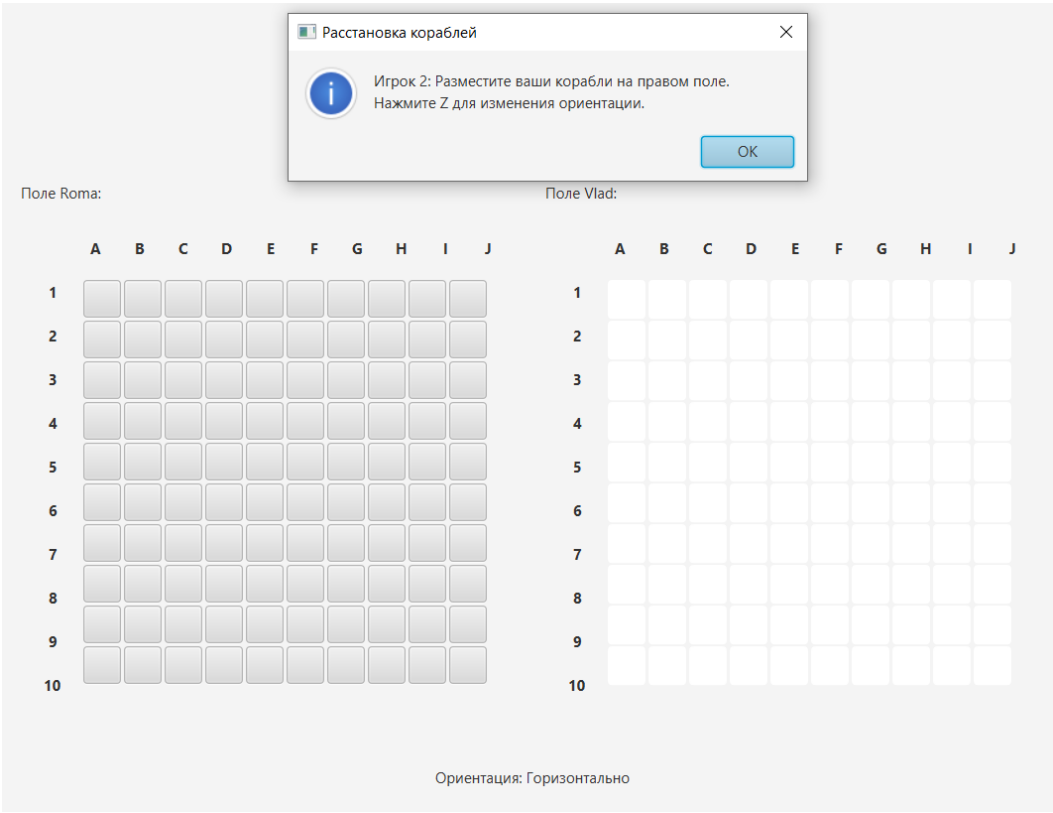


Рисунок 3 — Проверка скрытия поля

Помимо этого, существует сценарий атак, то есть когда игроки делают выстрелы по полям друг друга (рисунок 4).

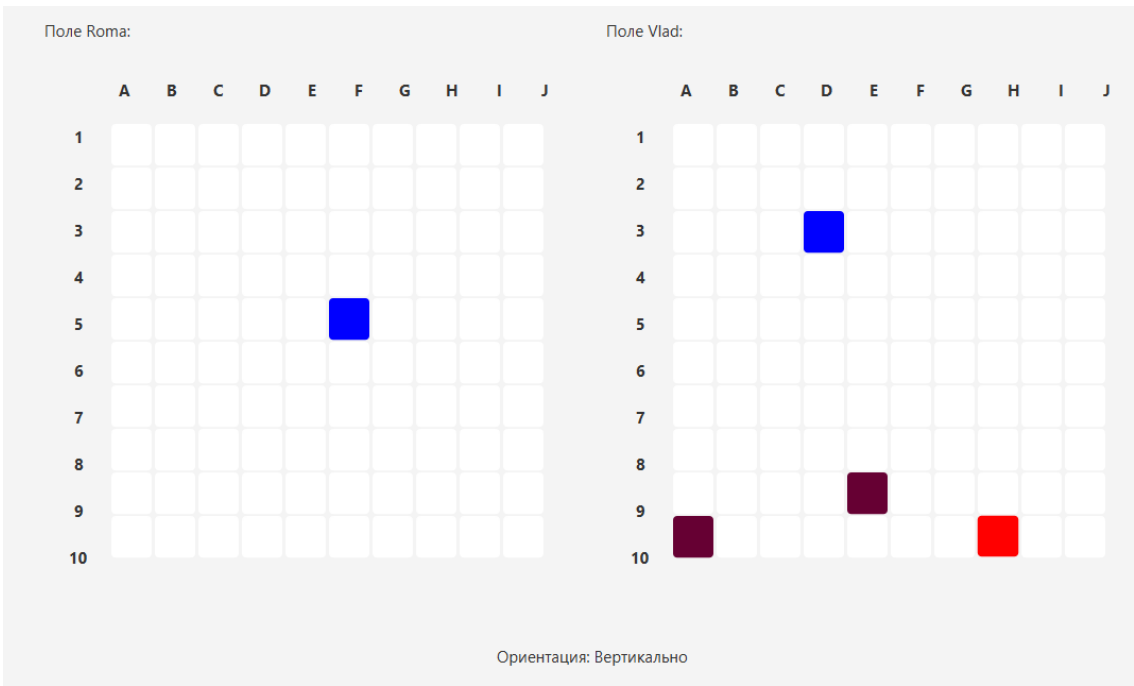


Рисунок 4 – Проверка отображения атак

Так же существуют состояние у кораблей: потоплен ли он, побит ли полностью, или же по нему осуществлен выстрел, а так же все ли корабли поставлены на поле и то что по ним нельзя совершить повторные выстрелы (Рисунок 5, 6)

```
3  import org.junit.jupiter.api.Test;
4  import static org.junit.jupiter.api.Assertions.*;
5
6  class ShipTest {
7
8      @Test
9      void testConstructor() {
10         Ship ship = new Ship(3);
11         assertEquals(3, ship.getSize());
12
13         assertFalse(ship.isSunk());
14     }
15
16     @Test
17     void testHit() {
18         Ship ship = new Ship(2);
19         ship.hit();
20
21         assertFalse(ship.isSunk());
22
23         ship.hit();
24     }
25 }
```

Рисунок 5 — Проверка регистрации состояние кораблей

```

3 import org.junit.jupiter.api.BeforeEach;
4 import org.junit.jupiter.api.Test;
5
6 import static org.junit.jupiter.api.Assertions.*;
7
8 class BoardTest { new *
9     private Board board; 37 usages
10
11     @BeforeEach new *
12     void setUp() {
13         board = new Board();
14     }
15
16     @Test new *
17     void testIsCellWithinBounds() {
18         assertTrue(board.isCellWithinBounds( x: 0, y: 0));
19         assertTrue(board.isCellWithinBounds( x: 9, y: 9));
20         assertFalse(board.isCellWithinBounds( x: -1, y: 5));
21         assertFalse(board.isCellWithinBounds( x: 5, y: -1));
22         assertFalse(board.isCellWithinBounds( x: 10, y: 5));
23         assertFalse(board.isCellWithinBounds( x: 5, y: 10));
24     }
25 }

```

Рисунок 6 — Проверка методов размещения кораблей и возможность атак

4.2 Тестирование сохранения и отображения статистики

В конце игры важно понимать, чтобы проанализировать свои ошибки и дальше развивать свои умения в игре морской бой было добавлено отображение кораблей в конце победы одного из игрока (рисунок 7), а также сохранение и отображение статистики (рисунок 5, 6 и 8, 9)

```

мая 19, 2025 12:02:24 AM game.BattleRecord save
INFO: Запись боя успешно сохранена: Roma vs Vlad | Победитель: Roma | Режим: PvP | Дата: 2025-05-19 00:02:24.459
мая 19, 2025 12:02:24 AM game.DatabaseManager initializeDatabase
INFO: Подключение к базе данных SQLite: jdbc:sqlite:./battlehistory/battlehistory.db
мая 19, 2025 12:02:24 AM game.DatabaseManager createTableIfNotExists
INFO: Таблицы успешно созданы
мая 19, 2025 12:02:24 AM game.DatabaseManager initializeDatabase
INFO: База данных SQLite успешно инициализирована

```

Рисунок 5 — Сохранение статистики

```

16 class DatabaseManagerTest { new *
17     private static final String TEST_DB_URL = "jdbc:sqlite:./battlehistory/test_battlehistory.db"; no usages
18
19     @Mock 10 usages
20     private Connection mockConnection;
21     @Mock 1 usage
22     private Statement mockStatement;
23     @Mock 15 usages
24     private PreparedStatement mockPreparedStatement;
25     @Mock 2 usages
26     private PreparedStatement mockSelectStatement;
27     @Mock 4 usages
28     private ResultSet mockResultSet;
29
30     @BeforeEach new *
31     void setUp() throws Exception {
32         MockitoAnnotations.openMocks( testClass: this ).close();
33
34         // Настроить моки
35         when(mockConnection.createStatement()).thenReturn(mockStatement);
36         when(mockConnection.prepareStatement(anyString())).thenReturn(mockPreparedStatement);

```

Рисунок 6 — Проверка инициализации базы данных и запись данных в нее

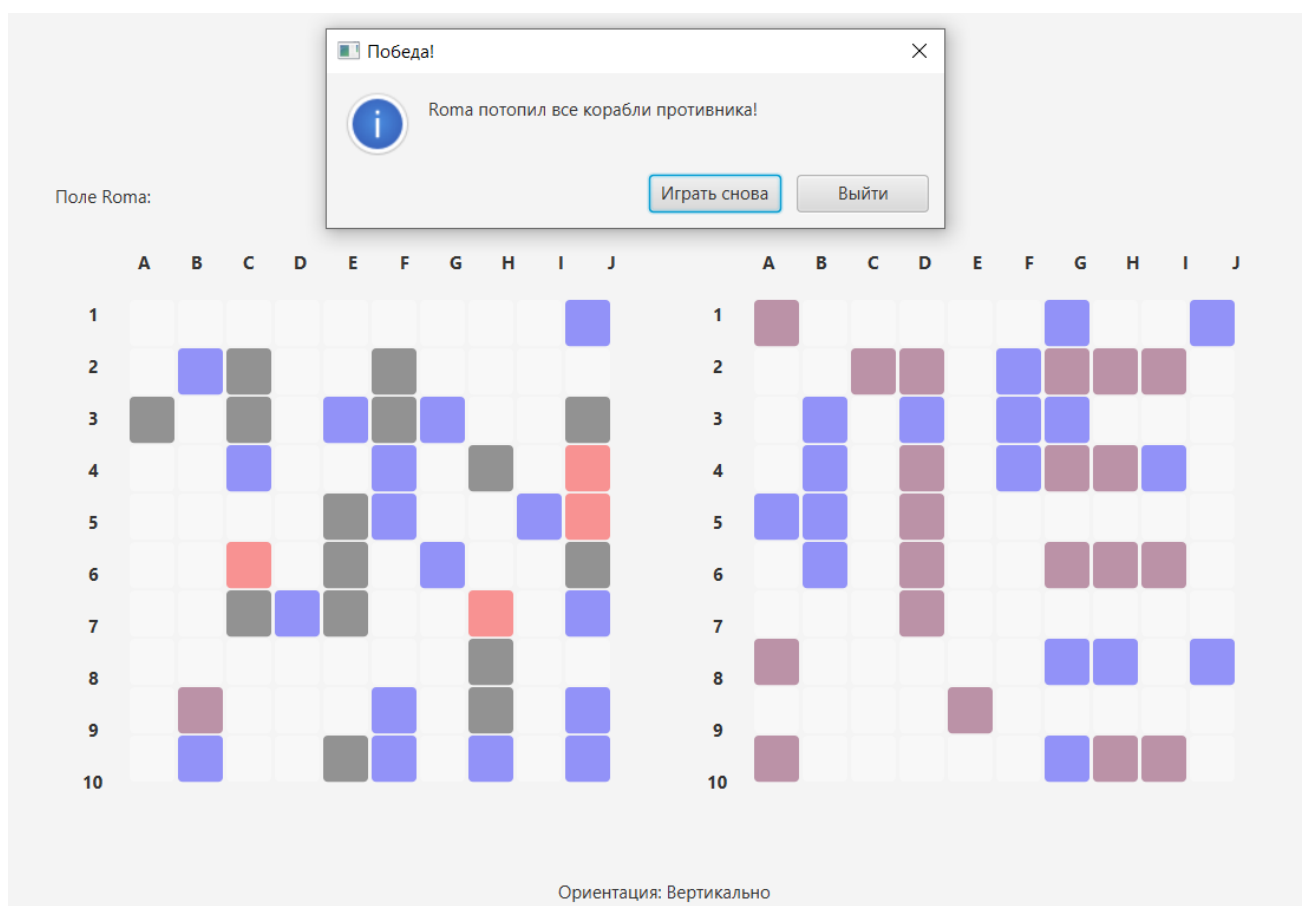


Рисунок 7 — Отображение кораблей после победы

Игрок 1	Игрок 2	Победитель	Режим игры	Дата боя
Roma	Vlad	Roma	PvP	2025-05-19 00:02:24.459
Vladislav	Aleksandr	Aleksandr	PvP	2025-05-06 18:11:18.56
Andrew	Компьютер	Andrew	PvE	2025-05-06 18:00:26.488
Andrew	Roma	Roma	PvP	2025-04-29 11:28:30.3
Danek	Компьютер	Danek	PvE	2025-04-29 09:14:28.464
Andrew	Компьютер	Andrew	PvE	2025-04-29 09:09:17.639
Andrew	Компьютер	Andrew	PvE	2025-04-29 09:06:45.024
Andrew	Kirill	Andrew	PvP	2025-04-25 20:44:28.175
Andrew	Danek	Danek	PvP	2025-04-25 20:09:50.246
Andrew	Danek	Andrew	PvP	2025-04-25 20:07:51.167
Andrew	Danek	Andrew	PvP	2025-04-25 20:04:55.357
Andrew	Danek	Andrew	PvP	2025-04-25 20:02:30.094
Andrew	Компьютер	Andrew	PvE	2025-04-24 13:20:24.502

Назад

Рисунок 8 — Отображение статистики игры

```

1 package game;
2
3 import org.junit.jupiter.api.Test;
4 import java.sql.Timestamp;
5 import static org.junit.jupiter.api.Assertions.*;
6
7 class BattleRecordTest {
8     @Test
9     void testBattleRecordCreation() {
10         Timestamp now = new Timestamp(System.currentTimeMillis());
11         BattleRecord record = new BattleRecord( player1: "Player1", player2: "Player2", winner: "Player1", gameMode: "PvP", now
12
13         assertEquals( expected: "Player1", record.getPlayer1());
14         assertEquals( expected: "Player2", record.getPlayer2());
15         assertEquals( expected: "Player1", record.getWinner());
16         assertEquals( expected: "PvP", record.getGameMode());
17         assertEquals(now, record.getBattleDate());
18     }
19
20     @Test
21     void testToString() {
22         Timestamp now = new Timestamp(System.currentTimeMillis());
23         BattleRecord record = new BattleRecord( player1: "Player1", player2: "Player2", winner: "Player1", gameMode: "PvP", now
24
25         String expected = String.format("Player1 vs Player2 | Победитель: Player1 | Режим: PvP | Дата: %s", now);
26         assertEquals(expected, record.toString());
27     }
28 }

```

Рисунок 9 — Проверка методов записи игры

4.3 Итоги тестирования

Таким образом, проведя тестирование, ошибок не было обнаружено, все функции выполнялись штатно и их результат можно было предугадать заранее. Тестирование проходило в несколько этапов: онлайн и оффлайн, что позволило избежать возможных казусов в ходе эксплуатации программы, а наличие прописанных unit-тестов заметно упростило модификацию готового продукта, позволяя оперативно реагировать на непредвиденные изменения в логике.

По результатам тестирования выявлено, что программа не имеет логических ошибок.

Данная программа тестировалась на компьютере следующей конфигурации:

1. Процессор: AMD Ryzen 5 7530U with Radeon Graphics 2.00 GHz;
2. ОЗУ: 16.00 Гб;
3. ОС: Windows 10 64-разрядная.

Заключение

В результате выполнения курсовой работы были улучшены навыки построения программ, использующие работу с базами данных, а также работу с графическими библиотеками по типу JavaFX. Получены навыки формализации, тестирования и отладки задач на компьютере, а также разработано приложение для игры в морской бой.

В ходе работы над курсовым проектом были улучшены навыки работы с Java, основными паттернами программирования, IntelliJ IDEA, SQLite.

В последствии программа может быть модифицирована для онлайн игры. А так же улучшение интерфейса путем добавления анимаций и звукового сопровождения.

					МИВУ.09.03.02-00.000 ПЗ	Лист
						30
Изм	Лист	№ докум.	Подп.	Дата		

Список литературы

1. Java полное руководство [Электронный ресурс]: лабораторный практикум/ — Электрон. текстовые данные.— Ставрополь: Северо-Кавказский федеральный университет, 2017.— 148 с.— Режим доступа: <http://www.iprbookshop.ru/75595.html>.— ЭБС «IPRbooks»

2. Лазицкас Е.А. Базы данных и системы управления базами данных [Электронный ресурс]: учебное пособие/ Лазицкас Е.А., Загумённикова И.Н., Гилевский П.Г.— Электрон. текстовые данные.— Минск: Республиканский институт профессионального образования (РИПО), 2018.— 268 с.— Режим доступа: <http://www.iprbookshop.ru/93382.html>.— ЭБС «IPRbooks»

3. Мирошников А.И. Архитектура систем управления базами данных [Электронный ресурс]: учебное пособие/ Мирошников А.И.— Электрон. текстовые данные.— Липецк: Липецкий государственный технический университет, ЭБС АСВ, 2018.— 94 с.— Режим доступа: <http://www.iprbookshop.ru/83189.html>.— ЭБС «IPRbooks»

4. Николаев Е.И. Базы данных в высокопроизводительных информационных системах [Электронный ресурс]: учебное пособие/ Николаев Е.И.— Электрон. текстовые данные.— Ставрополь: СевероКавказский федеральный университет, 2016.— 163 с.— Режим доступа: <http://www.iprbookshop.ru/69375.html>.— ЭБС «IPRbooks»